# CI/CD Pipeline for Machine Learning Lab Report

## Introduction

In this lab, I worked on building a complete CI/CD pipeline for a machine learning system. As I moved to further stages, I realized that machine learning systems are much more dynamic because they depend not only on code but also on data and model behaviour over time.

I understood that unlike traditional systems, a machine learning pipeline does not end after deployment. It requires continuous validation, monitoring, and retraining to remain reliable and operational. This lab helped me understand how each stage of the pipeline plays a part in building a stable and production-ready system.

Rather than treating each stage separately, we can see this pipeline as a continuous flow where every step connects to the next, like a complete lifecycle.

## Training Pipeline (train.py)

The pipeline begins with the training stage, where the model is built using the Iris dataset.

In this stage, the dataset is loaded and split into training and testing sets to ensure proper evaluation. A Random Forest model is then trained using predefined configuration parameters. This ensures that the training process remains consistent across different runs.

One concept that I learnt was crucial here is data versioning. By generating a hash of the dataset, we can track whether the data has changed between training runs. I realized that it is highly useful for reproducibility and debugging.

The trained model is saved along with metadata such as performance metrics, configuration, and data hash. This ensures that the model can be reused and traced in the later stages of the pipeline.

This part of the lab mainly focuses on building a reliable and reproducible model.

**Validation Pipeline (validate.py)**

Training alone is not sufficient to guarantee reliability, which makes validation an essential next step. After training, the model is not directly deployed. Instead, it first goes through a validation stage that acts as a quality control system.

Validation stage showed that multiple checks with different metrics are required before a model can be trusted or deployed for production.

The validation process includes four gates.

1.Schema validation ensures that the input data structure is correct.
2.Performance validation checks whether the model meets accuracy and F1 score thresholds.

3.The regression check ensures that the model is not worse than the previous version.

4.Fairness validation ensures that the model performs well across all classes.

Only if the model passes all these gates, it can move forward in the pipeline.

This made it clear that in real-world systems, deploying a model without proper validation can lead to serious issues.

**Drift Detection (drift_detect.py)**

Even after deployment, the model may not remain reliable if the data changes over time. Drift Detection stage focuses on detecting such changes.

Here, statistical techniques like PSI (Population Stability Index) and KL divergence are used to compare training data with production data. After implementing these concepts and observing the outputs, it became clear that higher values indicate greater data shift.

Based on these values, drift is classified into different levels such as slight or severe. If severe drift is detected, the system recommends retraining.

This helped me understand that machine learning models are not static and must adapt to changing real-world data.

**Continuous Integration (CI Pipeline)**

The CI pipeline automates the process of training and validation using GitHub Actions.

It is triggered whenever there are changes in code or data, or based on a schedule. The pipeline ensures that the model is trained and validated automatically without manual intervention.

One important concept here is job dependency. Validation runs only after training is completed, this makes sure that the workflow follows the correct sequence.

Another key aspect is using the commit SHA as the model version. This helps in tracking which version of the code produced a particular model.

Continuous Integration stage ensures automation, consistency, and reproducibility in the pipeline.

**Continuous Deployment (CD Pipeline)**

After validation, the model is deployed using a canary deployment strategy.

Instead of deploying the model to all users at once, it is first released to a small percentage of traffic(users). The system then monitors its performance in terms of accuracy and latency.

If the model performs well, it is promoted to full deployment. Otherwise, it is automatically rolled back to the previous version.

This approach reduces risk and ensures that unreliable models do not affect all users.

**Production Monitoring (monitor.py)**

The final stage of the pipeline is monitoring, which ensures that the model continues to perform well after deployment.

The system continuously tracks metrics such as accuracy and feature drift for any changes. If performance drops beyond a certain set threshold, retraining of the model is triggered automatically.

A circuit breaker mechanism is also implemented to prevent too many retraining attempts within a short time period.

During execution in notebook environment, a warning related to datetime.utcnow() was observed. I learnt that this is due to changes in newer Python versions and that it does not affect the functionality of the system.

This completes the pipeline by creating a continuous feedback loop.

**Issues Faced and Resolution**

**Issue 1: CI Pipeline Validation Failing Despite Correct Logic**

While implementing the CI pipeline, the validation step failed even though the logic appeared to be correct and terminal checks were passing successfully. This created confusion because there was no obvious error in the implementation.

After analyzing the issue, I realized that the problem was not with the logic but with the way path filters were defined. Initially, I used general patterns like **/*.py, assuming they would be acceptable.

However, the lab validator required exact patterns such as src/** and data/**. Once I updated the configuration to match these expected formats, the pipeline validation passed successfully.

This issue taught me that real-world systems may rely on strict configuration rules rather than flexible logic.

**Issue 2: YAML Formatting Errors**

During the implementation of CI/CD pipelines, some steps failed even though the commands themselves were correct. This was initially difficult to debug because the errors were not directly related to the logic.

The issue was caused by incorrect YAML formatting. I had used single-line commands instead of block format, which led to parsing errors.

After switching to block format using:

```
run: |
  command here
```

the issue was resolved.

This highlighted that YAML is highly sensitive to formatting, and even small structural mistakes can cause failures.

**Issue 3: Confusion Between UI Results and Terminal Output**

At one stage, the terminal showed that the pipeline was executing successfully, but the lab interface still displayed errors. This mismatch created confusion about whether the implementation was correct.

After investigating further, I understood that the lab environment validates based on expected output patterns and configurations, not just execution success.

By aligning the implementation strictly with the expected format, the issue was resolved.

This taught me that understanding system expectations is just as important as writing correct code.

**Issue 4: Difficulty in Understanding Drift Detection Metrics**

The drift detection stage introduced statistical concepts like PSI and KL divergence, which were initially difficult to understand.

At first, it was unclear how these values related to real-world model performance. The concepts seemed abstract and not directly intuitive.

By implementing the formulas step-by-step and analyzing the outputs, I gradually understood that higher PSI values indicate significant changes in data distribution.

This helped in identifying when a model may no longer perform reliably and when retraining is required.

This experience showed the importance of monitoring data changes in real-world machine learning systems.

**Key Learnings**

This lab helped me understand that CI/CD in machine learning is much more complex than traditional pipelines. Each stage plays a critical role in ensuring system reliability.

Training ensures reproducibility, validation ensures quality, drift detection ensures adaptability, CI ensures automation, CD ensures safe deployment, and monitoring ensures continuous performance.

The most important takeaway is that machine learning systems require a continuous feedback loop, and skipping any stage can lead to unreliable results.

**Conclusion**

This lab provided a complete understanding of how machine learning systems are built and maintained in real-world environments.

The most important thing I learned from this lab was that building a model is only one part of the process and that ensuring that the model continues to perform well after deployment is equally important.

By connecting all stages into a single pipeline, I was able to understand how automation, validation, and monitoring work together to maintain system reliability.

**Note: AI tools were used for understanding concepts, debugging issues, and improving the structure of the report.**